BioSync: Can Google Fit Write to Health Connect While BioSync Only Reads?

1. Direct Answer: Yes, It Is 100% Fine—With Two Important Trade-Offs to Know

Letting **Google Fit handle all writing to Health Connect** while **BioSync is strictly read-only** (`READ_STEPS`, `READ_DISTANCE`, `READ_TOTAL_CALORIES_BURNED`) is actually the **cleanest and safest Health Connect architecture**.

In fact, stopping BioSync from writing or deleting records in Health Connect immediately prevents **data feedback loops** (where BioSync writes steps, Google Fit reads them back, or BioSync overwrites Google Fit's synced wearable data).

However, before you rely *solely* on reading Health Connect, you need to understand **how Google Fit syncs to Health Connect under the hood** so you aren't surprised by its behavior.

---

2. How It Works (and the 2 Catch Points)

Catch #1: Health Connect Is NOT Real-Time (10–15 Minute Sync Delay)

- **How Google Fit writes:** Google Fit does **not** write every single step into Health Connect in real time as you walk. Doing so would drain the phone's battery through constant IPC/database writes. Instead, Google Fit batches its step records and flushes them into Health Connect periodically (typically every **10 to 15 minutes** in the background, or immediately when the user opens the Google Fit app UI).
- **What this means for BioSync:** If a user opens BioSync, walks 200 steps across the room while staring at your dashboard, and BioSync *only* reads Health Connect, **your step counter will look frozen at 0** until Google Fit performs its next background flush into Health Connect.

Catch #2: The User Must Have Google Fit Installed & Connected

- If a user installs BioSync on a phone that **does not have Google Fit installed** (or hasn't enabled Health Connect sync inside Google Fit settings), Health Connect will return `0` steps because Health Connect itself is just an encrypted local database—it

does not count steps on its own without a writer app like Google Fit, Samsung Health, or Fitbit.

---

3. The Best-Practice Hybrid Architecture (Read-Only Health Connect + Live Hardware Sensor)

To get the **100% accuracy of Google Fit** *and* **instant live step counting in your UI**, use this exact pattern without ever writing to Health Connect from BioSync:



Why This Works Perfectly:

1. **Zero Health Connect Conflicts:** BioSync requests **only** `READ_STEPS` (and `READ_DISTANCE` / `READ_ACTIVE_CALORIES_BURNED`). You delete all `insertRecords()` and `deleteRecords()` calls from BioSync.
2. **Instant UI Updates While Walking:** When the user walks with their phone in hand, your local `TYPE_STEP_COUNTER` hardware sensor increments immediately on screen.
3. **Automatic Reconciliation with Google Fit:** Whenever Health Connect has a higher total (for example, the user walked with a Wear OS watch, or Google Fit synced background steps while BioSync wasn't running), `max(healthConnectSteps, localSensorSteps)` automatically snaps up to Google Fit's total.

4. **Works Even Without Google Fit:** If a user doesn't have Google Fit installed, `healthConnectStepsToday` is `0`, so `max(0, localSensorSteps)` seamlessly falls back to your built-in hardware sensor!

---

4. Minimal Read-Only Code for `BioSyncHealthStepManager.kt`

Here is how clean your Health Connect manager becomes when BioSync only reads from Health Connect:

Kotlin

```kotlin
package com.example

import android.content.Context
import androidx.health.connect.client.HealthConnectClient
import androidx.health.connect.client.permission.HealthPermission
import androidx.health.connect.client.records.DistanceRecord
import androidx.health.connect.client.records.StepsRecord
import androidx.health.connect.client.records.TotalCaloriesBurnedRecord
import androidx.health.connect.client.request.AggregateRequest
import androidx.health.connect.client.time.TimeRangeFilter
import java.time.LocalDate
import java.time.LocalDateTime
import java.time.ZoneId

class BioSyncHealthStepManager(private val context: Context) {

    private val healthConnectClient by lazy {
HealthConnectClient.getOrCreate(context) }

    // Strictly READ-ONLY permissions — no WRITE permissions needed!
    val permissions = setOf(
        HealthPermission.getReadPermission(StepsRecord::class),
        HealthPermission.getReadPermission(DistanceRecord::class),
        HealthPermission.getReadPermission(TotalCaloriesBurnedRecord::class)
    )

    suspend fun hasAllPermissions(): Boolean {
        return try {
            val granted =
healthConnectClient.permissionController.getGrantedPermissions()
            granted.containsAll(permissions)
        } catch (e: Exception) {
            false
        }
```

```
    }

    /**
     * Reads the deduplicated daily step total written by Google Fit (or any
connected wearable)
     * from Midnight (00:00) today up to right now.
     */
    suspend fun readTodayStepsFromHealthConnect(): Long {
        return try {
            if (!hasAllPermissions()) return 0L

            val zoneId = ZoneId.systemDefault()
            val startOfDay =
LocalDate.now(zoneId).atStartOfDay(zoneId).toInstant()
            val now = LocalDateTime.now(zoneId).atZone(zoneId).toInstant()

            val response = healthConnectClient.aggregate(
                AggregateRequest(
                    metrics = setOf(StepsRecord.COUNT_TOTAL),
                    timeRangeFilter = TimeRangeFilter.between(startOfDay, now)
                )
            )
            response[StepsRecord.COUNT_TOTAL] ?: 0L
        } catch (e: Exception) {
            0L
        }
    }
}
```

---

5. Summary Checklist for Your App

1. **AndroidManifest.xml:** Remove `android.permission.health.WRITE_STEPS`, `WRITE_DISTANCE`, and `WRITE_TOTAL_CALORIES_BURNED`. Keep only the `READ_*` permissions.
2. **Health Connect Calls:** Always use `healthConnectClient.aggregate(AggregateRequest(metrics = setOf(StepsRecord.COUNT_TOTAL), ...))` rather than `readRecords()`. `aggregate()` automatically deduplicates overlapping step buckets if Google Fit writes both phone and watch steps.
3. **Live UI Formula:** Use `val finalSteps = maxOf(healthConnectSteps, localHardwareSensorSteps)` so the user gets live step updates while walking *plus* Google Fit's full daily total.